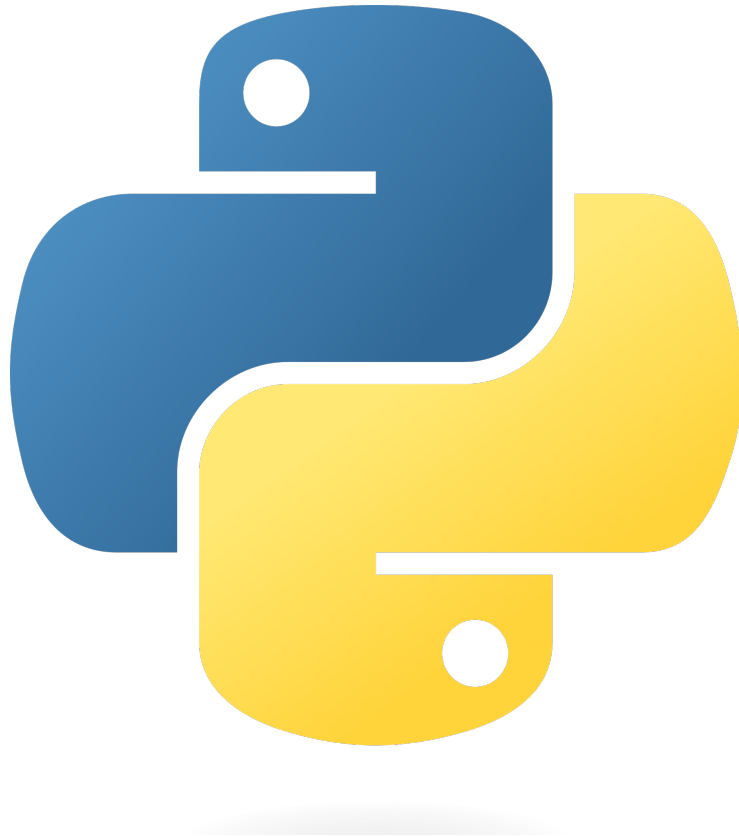


King of The Ocean

Python Programming Project



Gaël KERNINON and Clément GOMEZ

Cracow University of Technology, Poland 

June 2, 2026

Contents

1	Introduction	2
2	Game Architecture	3
2.1	Global Constants	3
3	Implementation	3
3.1	Entity Management	3
3.2	Collision Detection	3
4	The Levels	4
5	Execution and Results	5
6	Conclusion	5
.		

1 Introduction

This document serves as the project report for the *King of the Ocean* video game project, developed during the 2026 Erasmus+ semester at the Cracow University of Technology in Poland.

Following our previous work on historical cryptography and text processing, the main objective of this student project was to use the Python programming language with the Pygame library to build a functional 2D arcade game. The project focuses on object and entity management, real-time collision detection, and asynchronous event handling within a continuous game loop.

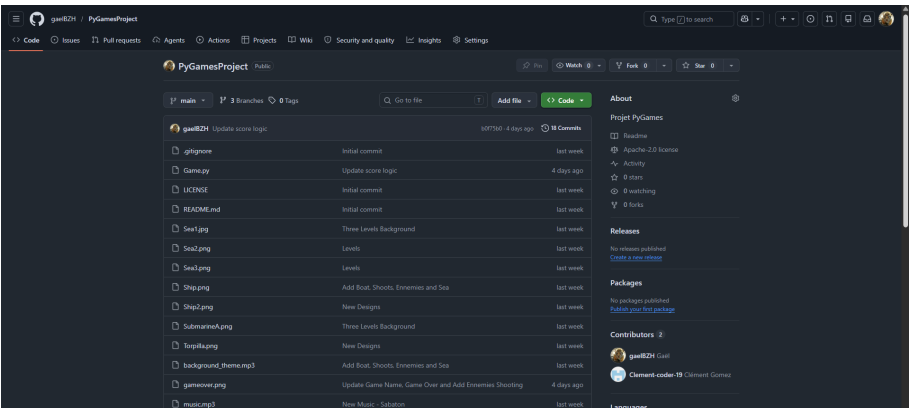


Figure 1: GitHub Repository of the Project

2 Game Architecture

The project is centralized within a single script (`Game.py`) that initializes the graphical environment, loads multimedia assets, and executes the main game loop.

2.1 Global Constants

To maintain code readability and allow for easy gameplay balancing, core parameters are defined as global constants. This includes display dimensions, player speeds, and entity limits.

```
1 GAME_NAME = "King of the Ocean"
2 SCREEN_WIDTH = 800
3 SCREEN_HEIGHT = 600
4
5 PLAYER_WIDTH = 250
6 PLAYER_HEIGHT = 70
7 PLAYER_SPEED = 2
8 ENEMY_WIDTH = 135
9 ENEMY_HEIGHT = 44
10 ENEMY_SHOOTRATE = 40
11 ENEMY_SPEED_MIN = 1
12 ENEMY_SPEED_MAX = 2
13 ENEMY_SPAWN_RATE = 105
14 MAX_ENEMIES = 6
15
16 BULLET_WIDTH = 8
17 BULLET_HEIGHT = 40
18 BULLET_SPEED = 1
19 ENEMY_BULLET_SPEED = 2
20 MAX_BULLETS = 3
21
22 LEVEL_1 = 0
23 LEVEL_2 = 20
24 LEVEL_3 = 50
```

3 Implementation

3.1 Entity Management

Rather than using complex Object-Oriented Programming (OOP) class structures, the game utilizes Python lists and dictionaries to track entities dynamically.

- **Player:** Managed via a single `pygame.Rect` object mapped to the keyboard's left and right arrow keys.
- **Enemies:** Stored in an `enemy_list` array as dictionaries containing their `Rect`, speed, firing state, and spawn side. Enemies spawn randomly on the left or right edges and traverse the screen horizontally.
- **Projectiles:** Tracked in `bullet_list`¹ and `enemy_bullet_list`².

3.2 Collision Detection

Collision detection relies on Pygame's built-in `colliderect()` method. During each iteration of the game loop, the engine checks for intersections between:

1. Player projectiles and enemy `Rects` (resulting in score increments and entity deletion).
2. Enemy projectiles and the player `Rect` (resulting in a Game Over state).

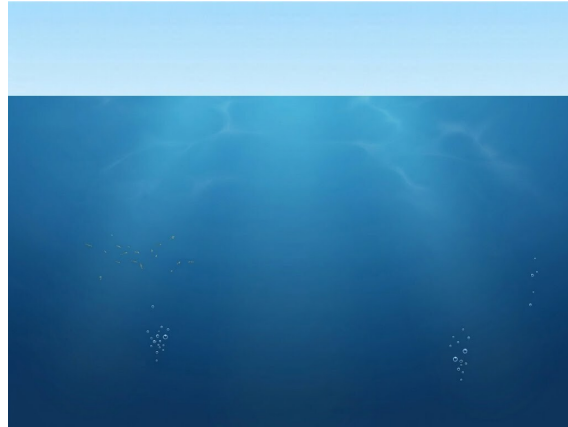
¹Player's Torpedoes

²Enemy's Torpedoes

4 The Levels

To maintain player engagement, the game implements a progressive difficulty system based on the player's score. The environment and gameplay constraints shift across three distinct levels:

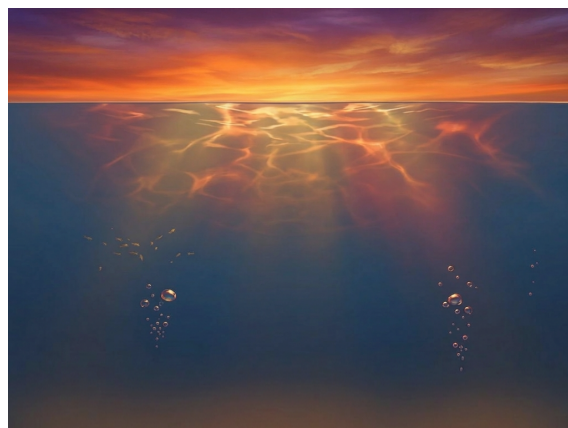
- **Level 1 (Score ≥ 0):** Default sea (Sea1.jpg). Enemies move slowly, and the player is restricted to 2 simultaneous projectiles.



- **Level 2 (Score ≥ 20):** Coral Sea (Sea2.png). Max enemies increased to 6, enemy speeds increase, and the player can fire up to 5 projectiles.



- **Level 3 (Score ≥ 50):** Sunset Sea (Sea3.png). Max enemies increased to 8, high enemy fire rate (80%), and fast pacing.



5 Execution and Results

Upon launching `Game.py`, the `pygame.mixer` initializes the background audio (`music.mp3`), and the primary game loop begins listening for `pygame.KEYDOWN` events.



Figure 2: In-game screenshot showcasing Level 3 gameplay, UI overlay, and entity sprites

The graphical user interface overlays the current Score and Level in the top-left corner, while rendering available torpedo ammunition in the top-right corner. When the player is hit, the loop transitions to a Game Over state, displaying the `gameover.png` asset and halting movement until the user presses 'Q' to quit or 'P' to restart.

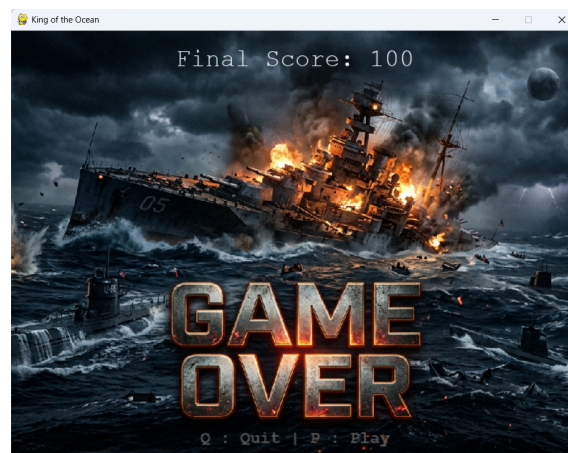


Figure 3: Game Over Screen

6 Conclusion

The *King of the Ocean* project was an excellent exercise in applied Python programming for multimedia environments that shows how Python can be easily used for Video Games programming with PyGame module.

References

- [1] Pygame Community, *Pygame Documentation*. Available at: <https://www.pygame.org/docs/>
- [2] Python Software Foundation, *Python Language Reference*. Available at: <https://docs.python.org/3/>

